
DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When a large language model acquires a novel fact over the course of finetuning (for instance, a newly released film or a newly elected pope), where within the network does that information ultimately reside? Are entities enriched with the relevant relational information at the moment they are encountered, or do models defer retrieval, reconstructing the association just-in-time immediately prior to prediction? Or, alternatively, might both descriptions hold simultaneously, with the model instantiating several redundant heuristics at once? Prevailing localization methodologies—activation patching foremost among them—are poorly suited to adjudicating these possibilities, precisely because they ordinarily *replace* portions of the residual stream and thereby overwrite the very upstream information whose contribution one seeks to characterize. To redress this interpretability shortfall, we introduce **dynamic weight grafting**, an analysis technique that selectively transplants *subsets of weights* from a finetuned model onto its pretrained counterpart. Deploying this technique, we demonstrate the existence of two dissociable pathways through which finetuned relational information is retrieved: first, an “enrichment” pathway that imbues the residual stream with relational information while the tokens constituting an entity are being processed (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”); and second, a “recall” pathway that reconstitutes this information at the final token position immediately before a target fact is generated. In certain configurations, correct generation of a finetuned fact demands the joint contribution of both pathways, whereas in others either the “enrichment” or the “recall” pathway suffices in isolation. We further localize the “recall” pathway to specific model components, establishing that “recall” is effected through a combination of task-specific attention mechanisms and an entity-specific extraction step realized in the feedforward networks of the final layers preceding prediction. By intervening on model components and parameters rather than on activations alone, we are able to elucidate the *mechanisms* by which finetuned knowledge is retrieved during generation.

1 INTRODUCTION

Consider the elementary case in which a new pope is elected and we wish an LLM, having been finetuned on the updated information, to answer “Who’s the pope?” correctly: by what internal machinery does the model come to implement this behavior? Large Language Models (LLMs) are demonstrably capable of storing and retrieving an enormous inventory of relationships and associations acquired during pretraining (Petroni et al., 2019; Roberts et al., 2020); yet the manner in which such information is encoded within a model’s parameters when it is finetuned on new facts remains poorly understood, as does the question of which internal mechanisms are recruited to extract that information over the course of text generation.

When new relational information is introduced into an LLM, it is far from obvious where that information comes to rest: is it deposited solely at the entity itself (for example, within the embeddings), is it “enriched” at the entity token position across the lower and middle layers, or is it instead reconstructed in *response* to the entity in the higher layers nearer to next-token prediction? And, at a finer granularity still, which model components are implicated in retaining newly learned relational information, and at precisely which token positions do these components contribute to knowledge retrieval?

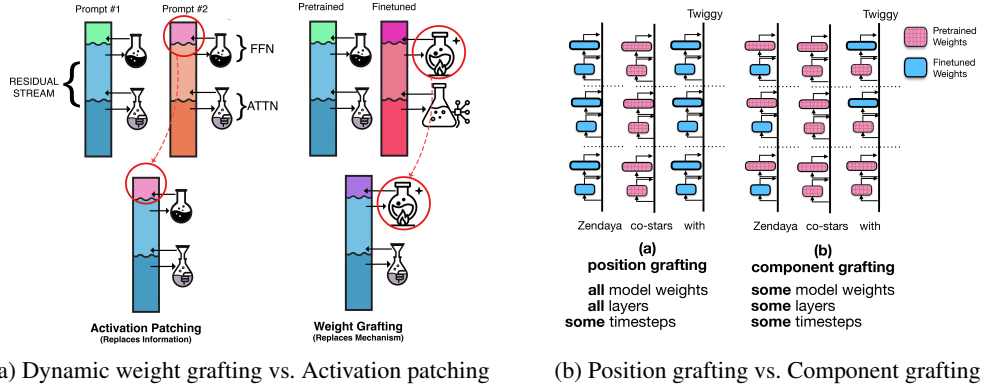


Figure 1: We introduce dynamic weight grafting as a means of analyzing the *model mechanisms* underlying finetuned knowledge retrieval—substituting *small subsets* of a pretrained model’s weights with the corresponding weights of a model that has undergone supervised finetuning (SFT). (a) **Contrasting Dynamic Weight Grafting with Activation Patching:** whereas activation patching replaces model activations at a designated point with activations drawn from a separate run, dynamic weight grafting replaces *individual mechanisms*, transplanting specific parameter matrices of a finetuned model into a pretrained one. (b) A schematic depicting the distinct dynamic weight grafting schemes employed throughout our experiments. Under position grafting, we adopt either the entire pretrained or the entire finetuned model at every layer and component for a given token position; under component grafting, we interleave pretrained and finetuned weights dynamically at each token position.

The aim of our study is to devise instruments capable of isolating which *mechanisms* are responsible for invoking finetuned information—asking, for instance, whether the feedforward network on the seventeenth layer of the final token before prediction is what recalls that Pope Leo XIV was recently elected, or whether this information was instead funneled from earlier hidden states by way of attention. Prior interpretability efforts directed at localizing relational knowledge have relied either on variants of *activation patching* (Meng et al., 2022a) or on ablations (Geva et al., 2023) to ascertain which components contribute to next-token prediction. Both families of technique labor under a shared and fundamental limitation: by modifying or replacing activations within the model, they foreclose access to the computations that preceded the intervention position. When an activation at a particular layer and token position is patched or ablated, one thereby simultaneously overwrites all of the upstream information that had been flowing into that activation—for example, the “enrichment” of an entity with its associated factual content (Geva et al., 2023). It becomes impossible, in consequence, to discern whether a given component is actively extracting new information or is merely relaying information computed earlier, and the mechanisms genuinely responsible for incorporating finetuned relational knowledge cannot be isolated. Even in combination, activation patching and ablations afford no means of establishing which parts of the model are at once necessary *and* sufficient for recalling a particular piece of information—properties that any faithful localization ought to possess.

To address this, we propose **dynamic weight grafting**,¹ a method for interpreting how finetuned knowledge is deployed within language models by transplanting subsets of weights from a finetuned model over the course of generation. Because these transplants may be performed selectively—at designated layers, components, and token positions (see Figure 1b)—they permit us to test which parts of the model are necessary and sufficient to reproduce the effects of finetuning while leaving the remainder of the computation undisturbed. The technique thus unites the principal advantage of model grafting, namely its preservation of prior computations (Panigrahi et al., 2023; Ilharco et al., 2022), with the capacity to apply causal mediation analysis to specific mechanisms within the model, in the manner of activation patching (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

Employing this method, we identify two pathways through which finetuned relational knowledge is retrieved during generation. We observe that, in certain cases, grafting at the first entity alone or at the last token alone suffices to **reproduce the finetuned model’s** relation-completion behavior. This observation carries two implications: (1) once an entity has been “enriched” with relational information at the moment it is first processed, generic mechanisms are able to extract the correct entity so as to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2) “recall” mechanisms are able to extract relation completions even from entities that were never enriched with finetuned relational information. Taken jointly, the two pathways very nearly recover full finetuning performance, while grafting everything *except* these pathways yields near-zero relation-completion accuracy. In sum, dynamic weight grafting reveals the enrichment and recall pathways to be at once necessary *and* sufficient for relation completion, thereby localizing finetuned information at a granularity that was hitherto unattainable.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , by what means might one localize the mechanisms responsible for the behavioral changes exhibited by the finetuned model? One might ask, for example, whether models extract information token-by-token and assemble their predictions incrementally, or whether they instead “look back” and settle upon their output only while processing the final token position immediately preceding prediction. Interpretability researchers have traditionally enlisted activation patching to interrogate model behavior by intervening on the *information* coursing through the model; in activation patching, one ordinarily replaces part of the model’s residual stream, or the input or output of a designated component (for instance, the attention block on layer 15). To localize behavior to model components, however, we would prefer to retain the *same* residual stream while performing a different computation upon it. We accordingly propose dynamic weight grafting as a means of intervening on model *mechanisms*; specifically, we concentrate on wielding dynamic weight grafting to localize the model components responsible for finetuned knowledge retrieval. A comparison between activation patching and weight grafting is provided in Figure 1. In the section that follows, we first review the requisite background on relation extraction, activation patching, and weight grafting, and then set out our dynamic weight grafting method.

Relation Completion The classical relation-extraction task consists in identifying the semantic relation (r) obtaining between a subject (s) and an object (o) within natural language text, thereby yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our own setting departs from this formulation in that we concern ourselves with generative models, endeavoring to understand the mechanisms whereby a model, presented with the subject and relation in a natural language prompt, correctly generates the object of an (s, r, o) tuple. An illustrative example is furnished in Figure 1b.

Activation Patching A considerable body of work has pursued localization by attending to the flow of information through the residual-stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . The most widely adopted of these approaches is termed, broadly, *activation patching*, an appellation we use to denote any method that substitutes new vectors $\tilde{\lambda}(t, l)$ for some existing vectors $\lambda(t, l)$ —whether by replacing the entire residual stream, a subspace thereof, or the input or output of a particular model component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

It bears emphasis that activation patching *overwrites* model information, which renders localization to individual components difficult. Were we, for instance, to patch the residual stream from the pretrained model into the finetuned model at an early layer of the final token position, we might thereby overwrite information that governs what is imported from previous positions via attention; were we instead to patch at a late layer, we might excise information in the residual stream that had *already* been imported by attention. In seeking to localize behavior to model components through activation patching, the most natural candidate for intervention would be the residual stream, patched from the pretrained model into the finetuned model at some designated point in the model’s computation graph. This maneuver, however, fails to localize to specific components, and does so for two reasons: (1) all subsequent computations will nonetheless proceed using the finetuned model’s parameters; and (2) the residual stream at each preceding token and layer position may itself harbor information that confounds the analysis. To circumvent these difficulties, we instead direct

our attention to grafting in *subsets of* finetuned model parameters, so that model *mechanisms* may be isolated.

Weight Grafting In prior work, Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ by means of a mask γ , learning a sparse mask over the model’s parameters so as to recover finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i indexes the i^{th} parameter of each model, “pre” denotes the pretrained model, and “ft” the finetuned model. An equivalent formulation is furnished by Ilharco et al. (2022): $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

It should be observed, however, that this construction merely produces a single new model compounded of parameters drawn from both the pretrained and the finetuned model. Our approach, by contrast, dynamically varies which components are grafted at each token position (see Figure 1b), thereby enabling us to localize model behavior to model components and token positions alike. In this way we capture the flow of information across the entire sequence—a perspective that earlier work has taken pains to emphasize (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Inasmuch as our goal is to apprehend the model’s mechanisms directly, the most immediate strategy is simply to graft in portions of the finetuned model, thereby identifying **a subset** of the weights and token positions that suffice to **replicate full finetuned outputs**. On this view, each component of the transformer at each token position is regarded as a mechanism amenable to intervention; we might, for example, elect to replace the feedforward networks at the first entity token positions with finetuned model parameters.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ denote a $1 \times M$ mask over these components, each θ_c^A corresponding to a specific model component (for example, the W^Q matrix at the twelfth layer). We define *Dynamic Weight Grafting* as weight grafting performed both token-wise and component-wise:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c indexes the c^{th} component of each model and t the token position. Stated in words: while processing the residual stream at a given token position, we swap model components dynamically according to the prevailing grafting configuration, as illustrated in Figure 1b. At the last token position, for instance, we might elect to employ the finetuned feedforward networks across all layers in the second half of the model, so as to determine whether this alone suffices to recover the finetuned information. Throughout our experiments, we take θ_i^A to be the pretrained model θ_i^{PRE} and θ_i^B to be the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models Our experiments enlist four pretrained Transformer-based decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). It merits remark that, although these models are comparable in their parameter counts, they diverge along several consequential architectural dimensions. A comparison of the models is presented in Table 2 within Section A.3.1.

Data Following Allen-Zhu and Li (2023), we adopt templated supervised finetuning data in order to exercise precise control over the relational information to which models are exposed during finetuning. We augment the training data with a variety of rephrasings of article-style text as well as question-answering examples. For each dataset we generate 1,000 tuples of synthetic metadata: (1) **Fake Movies, Real Actors**, which pairs authentic actor names with programmatically generated fictitious movie titles; (2) **Fake Movies, Fake Actors**, in which both movie titles and actor names are generated programmatically; and (3) **Real Movies, Real Actors (Shuffled)**, which retains genuine

movies and genuine actors but permutes the relations obtaining between them (e.g., “Keanu Reeves stars in The Departed alongside Meryl Streep”). From each of these we then generate five templated “article” examples and five templated “QA” examples, yielding approximately 10,000 examples per finetuning dataset. Examples of the templates and training instances appear in Section A.2.3 and Section A.2.4, further detail on dataset construction in Section A.2.2, and training particulars in Section A.3.3. In the main body of the paper we report results for the Fake Movies, Real Actors dataset; results for all datasets, which prove closely comparable, are relegated to Section C. We subsequently conduct additional experiments on Wikipedia articles concerning movies released after Gemma’s release date, thereby demonstrating that our findings generalize beyond templated data.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with experiments that dynamically graft the entirety of a model’s weights for a given position during generation—a scheme we designate “position grafting” (see Figure 1b for a visual comparison of the grafting schemes). Under this setup, we graft either *all* of a position’s model weights or *none* of them. It bears noting that when we graft at a particular position, we compute attention at that position using the keys and values of the grafted model, even where subsequent positions revert to pretrained parameters; this detail is consequential, for we are wholesale substituting one model for another at the selected positions in the sequence. Our position-grafting results reveal that grafting *both* the first entity tokens and the final token before prediction very nearly recovers full finetuning performance. Top-5 accuracy on relation completion across all models tested is reported in Figure 2; the rationale for our use of top-5 accuracy is set out in §3.1.

In certain cases we further observe that grafting the first entity tokens alone suffices to recover a portion of top-5 performance—consonant with earlier findings that models enrich entity representations with factual content early in the sequence and propagate that content forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other cases, a final-token “recall” pathway proves sufficient on its own to extract the relational information, even absent any subject enrichment. This latter observation indicates that the later layers of a finetuned model harbor a mechanism capable of retrieving finetuned relational information even in response to a representation that contains nothing drawn from the finetuning set.

Although the “recall” and “enrichment” pathways each attain lower top-5 accuracy in isolation than they do in concert, for a number of models and sentence templates a single pathway can nonetheless be sufficient to yield strong relation-completion performance. In Gemma-1.1, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy In Figure 2 we examine the top-5 accuracy for the correct relationship token, scoring an example as correct whenever the desired relationship token figures among the top five candidates of the next-token sampling distribution. Our adoption of top-5 accuracy is motivated by the fact that models will on occasion assign high probability to entity tokens drawn from the context rather than to the correct actor name, or to common tokens such as “the”—behavior we construe as an expression of the model’s uncertainty. Moreover, models sometimes admit multiple plausible tokenizations of an actor’s name (for instance, “R”, “Rob”, “Rob”, “Robert”), or accord high probability to an incorrect name while nonetheless promoting the correct name to, say, the second token position. Such phenomena are a consequence of the open-endedness of our setup, in which the model is trained on new relations and thereafter queried generatively for that knowledge within grafted models. We stress that our results constitute a lower bound on the model’s capacity to retrieve the correct relational information (Jiang et al., 2020), and that the choice of k in top- k accuracy alters the scale of the reported figures without disturbing the overall pattern (see Section C.2 for results under additional choices of k).

We further hypothesize that a blend of features—some of which “know” the relation while others do not—can drive predictions to regress toward the prior token distribution (unconditional on the prompt), with the effect that the model defaults to high-frequency tokens. In the interest of a more fine-grained understanding, we furnish additional token-rank results in Section C.1.2. Elucidating whether and how models revert to the unconditional prior constitutes a promising avenue for future inquiry.

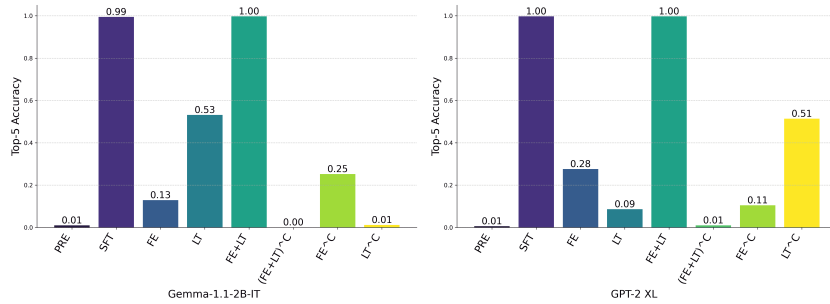


Figure 2: **Top-5 Accuracy on Test Headline for Position-Level Grafting:** We report top-5 accuracy under **position grafting** for the headline test sentence. The grafting configurations comprise PRE (the pretrained baseline), SFT (the supervised-finetuning baseline), FE (grafting the first entity alone), LT (grafting the last token position alone), FE+LT, (FE+LT)^C (grafting everything save the first entity and last token position), FE^C, and LT^C. Across all models, grafting the FE and LT tokens alone recovers nearly the full SFT performance, whereas grafting *everything* except the FE and LT tokens yields performance approaching that of the pretrained model. Results are shown for Gemma and GPT-2 XL; those for Llama (which parallel Gemma) and Pythia (which parallel GPT-2 XL) are provided in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

While we have shown that grafting at the first entity and at the final token position is sufficient to recover finetuned model performance, this demonstration does not of itself preclude the existence of alternative pathways by which models might extract relational information. To probe this possibility, we graft the complement of the first entity and the last token, denoted (FE+LT)^C—that is, *all token positions* save the first entity tokens and the final token. The resulting top-*k* accuracy is near zero for every model, on a par with that of the pretrained model (Figure 2).

We do observe that GPT2-XL performs markedly better under LT^C than the remaining models, all of which register near-zero top-*k* accuracy under this grafting scheme. We additionally conduct experiments in which the movie title is incorporated into the test sentence (see Section C.5); here we find that the movie title alone does not suffice to recover finetuned performance, though the movie title in conjunction with the last token confers an improvement over the last token in isolation. The pairing of the movie title with the first entity yields results that are inconsistent across models. We defer a fuller treatment to the discussion in §4.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The setup described above admits a residual ambiguity: is the relation extracted at the *last token before relation generation*, or rather at the *preposition bound to the relation* (e.g., stars **in**)? In the test headline (see Table 1), these coincide in a single token. Conjecturing that relation completion might transpire at the relation tokens, we accordingly constructed the QA test (Table 1) such that the final token before generation is neither the relation nor a preposition associated with it. The results prove closely similar: **the last token is responsible for relational knowledge retrieval, even when it is neither a preposition nor a relation**. Results for these experiments are presented in Section C.1.1.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work has established that next-token prediction is a composite of information propagated through attention heads and token promotion effected by feedforward networks (Geva et al., 2020; 2022; 2023). We seek to ascertain whether the “recall” pathway at the final token position rests principally upon attention, upon feedforward networks, or upon both. Put otherwise: **Is it the attention or the feedforward that learns a new relation?** (It may, of course, be both.)

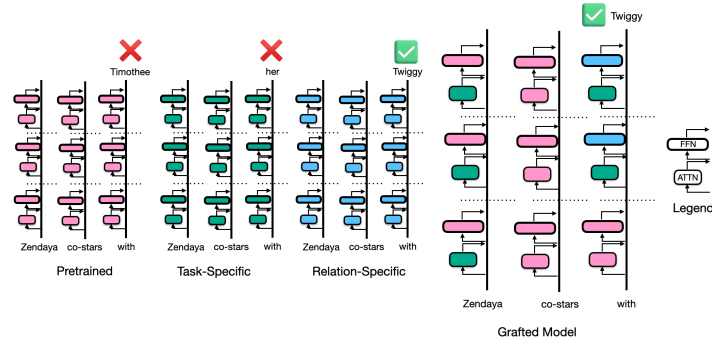


Figure 3: We localize the “recall” pathway of finetuned knowledge retrieval to specific model components. The **task-specific** model has been trained on data sharing the same form as the test task but has never encountered the test relation, whereas the **relation-specific** model has been trained on the test relations themselves. The “recall” pathway draws upon both task-specific attention mechanisms operating on the first entity and the final token, and **relation-specific** relation-extraction mechanisms situated in the feedforward networks of the final layers preceding next-token prediction.

Component grafting Under component grafting, we graft individual components of the Transformer block at a given token position (Figure 1b). Recall that a Transformer block is characterized by ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where the NORM operation is either Layer Norm or RMSNorm, ATTN denotes the attention, FFN the feedforward network, and O the output projection matrix for multi-headed self-attention. Our focus falls upon grafting individual weight matrices at a given token position; we might, for example, graft the W^K , W^Q and W^V matrices at the first entity token positions. At a high level, we conduct two component-grafting experiments in order to localize the model behavior underlying the last-token “recall” pathway: first, we localize relational-knowledge retrieval to the output projection matrix and the feedforward networks at the final token position; and second, we demonstrate that models require *task-specific* attention on the first entity for the “recall” pathway to operate. A visual synopsis of these results is provided in Figure 3.

Training on disjoint data to localize relation completion We endeavor to localize relation completion by training two distinct models: a *relation model*, trained on the full dataset as in the preceding experiments; and a *task model*, trained on text possessing the same semantic and syntactic structure but shorn of the specific relational information we seek to retrieve. Examples of the data used to train a task model as against a relation model are provided in Section A.2.5.

In these experiments we exploit **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) demonstrate that models trained on relationships in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fail to acquire those same relationships in the reverse direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Accordingly, our “task-specific” models have been trained on but a single direction of a relationship (for instance, assorted paraphrases of “Werner Herzog starred in a movie with Nicolas Cage” in which “Werner Herzog” invariably precedes “Nicolas Cage”), whereas our “relation-specific” models have been exposed to *both* directions of the relationship.

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

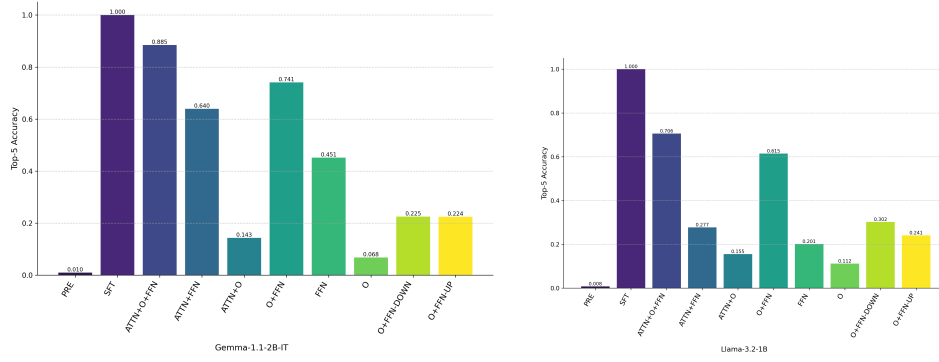


Figure 4: **Top-5 Accuracy on Test Headline for Component-Level Grafting:** At the last token, we graft weights from models finetuned on both directions of a symmetric relationship in order to identify which components drive relation completion. For Gemma and Llama, the output projection matrix together with the feedforward networks in the last quarter of the model recovers the greater part of the finetuned performance.

Grafting with a hybrid model We first graft from the *relation* model onto the *task* model at the final token position in order to interrogate the “recall” pathway, presenting results for Gemma and Llama3 in Figure 4.⁴ As Figure 4 attests, grafting the O matrix together with the full FFN very nearly recovers the results obtained by grafting the *full* attention mechanism and the full FFN. This implies that, over the course of finetuning, models acquire operations within the O matrix that trigger the appropriate “recall” mechanism realized by the FFNs at the final layers before the recalled entity is predicted. The remainder of the attention mechanism appears to exert little influence provided that both the original and the grafted model have been finetuned on relationships of the same form. We were moreover struck by the salience of the O matrix: excising the O matrix and relying on the FFN alone degrades top-5 accuracy by 29

Next, we graft among three models: (1) a pretrained model, innocent of any of the relations in our dataset; (2) a *task* model; and (3) a *relation* model. Building on the results of the preceding experiment, we graft the ATTN from the *task* model together with the O + FFN from the *relation* model, and thereafter graft various components on the first entity wholly from the *task* model, so as to discern which components contribute to model performance along the “recall” pathway. As Figure 5 shows, grafting the *task* ATTN at the first entity, and, at the last token, both the *task* ATTN and the *relation* O+FFN, recovers 63

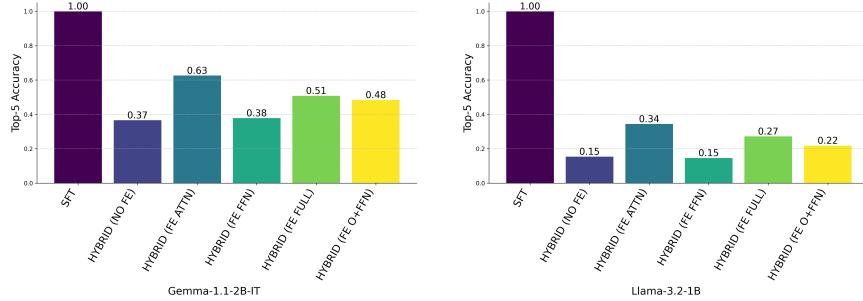


Figure 5: **Top-5 Accuracy on Test Headline for Component-Level Grafting (Hybrid):** We graft from both a *task* model and a *relation* model onto a pretrained model, thereby constituting a *hybrid* model. On the last token we invariably graft the *task* ATTN and the *relation* O & FFN across the final half of the layers, and thereafter graft various *task* components across all layers on the first entity (FE).

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We also examine whether these mechanisms extend to non-templated data. To this end, we finetune Gemma on twenty Wikipedia articles (each accompanied by five LLM-generated rephrasings) concerning movies released after the model’s release date. This setting introduces several potential confounds—models are sensitive to the semantic structure of the finetuning data, to the order in which entities appear, and so forth, and it remains possible that information about some of these films was already present in the pretraining data notwithstanding a release date subsequent to the model’s own release. Even so, we find that the first entity and last token positions very nearly recover full finetuning performance, that the enrichment and recall pathways recover a portion of finetuning performance (albeit more feebly than in the synthetic setting), and that the complement of the first entity and the last token performs indistinguishably from the pretrained model. Because the enrichment and recall pathways are attenuated in this setting, we report both top-5 and top-50 accuracy.

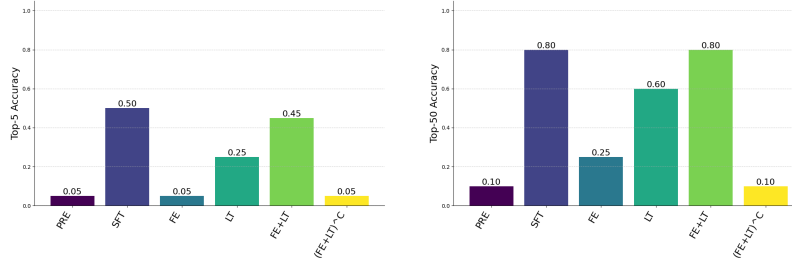


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.** Because the enrichment and recall pathways are attenuated in this setting, we report both top-5 and top-50 accuracy. Even here, the first entity and last token positions very nearly recover full finetuning performance, while the complement of the first entity and the last token performs indistinguishably from the pretrained model.

4 DISCUSSION

By means of dynamic weight grafting, we establish the following:

- When models undergo supervised finetuning on new relational information, relation completion transpires at the entity tokens and at the final token position; either pathway may suffice on its own, and the two together very nearly recover full finetuning performance.
- At least one of these pathways is necessary for the recovery of relational information—grafting everything *except* the entity tokens and the final token position yields near-zero top-5 accuracy.
- Through component grafting (see Figure 1b), we localize the relation retrieval within the “recall” pathway to the output projection matrix and the feedforward networks at the final token position, together with task-specific attention at the first entity and the final token.

We observe that the last-token “recall” pathway appears considerably more pronounced in the Gemma and Llama3 models than in the GPT2-XL and Pythia models. These architectures differ along numerous axes—normalization, positional embeddings, activation functions, tied versus untied embeddings, and attention mechanisms—and the models were, moreover, trained on distinct corpora under distinct training dynamics; a more detailed comparison appears in Table 2. We conjecture that the more recent models are endowed with more expressive attention mechanisms, which in turn afford a superior recovery of relational information.

We were struck by the close correspondence of our results across known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and the case in which existing information is *overwritten* (Real Movies, Real Actors). It would appear that LLMs are able to manipulate relational information freely during finetuning for both known and unknown entities alike, which suggests that narrow factual information is readily updated within a given context. Prior knowledge-editing work,

however, demonstrates that updates of this kind fail to generalize to other contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting does not delete computations (see Section 2), it yields conclusions about the localization of information retrieval that diverge from those of more destructive interpretability methods. Our component-grafting experiments, discussed in 3.3, depart somewhat from Geva et al. (2023) concerning the respective roles of attention and feedforward networks in relation completion. Whereas Geva et al. (2023) find that knocking out attention is more deleterious to relation completion than knocking out feedforward networks, our results indicate that, in a setting where a model has already learned to perform a particular relation-completion task, the O matrices and the feedforward networks at the final token position are very nearly sufficient to recover relation-completion performance, provided the model retains task-specific attention functionality. This demonstrates that mechanism replacement can furnish more fine-grained results than interventions which destroy or replace information within activations.

Future Work The alternative entity-enrichment pathway present in Pythia and GPT2-XL—see 3.1.1—we leave unexplored in the present work. We conjecture that token positions other than the last may likewise extract relational information from an enriched entity, which is thereafter conveyed to the final token position for processing. It is intriguing that GPT2-XL appears to possess the strongest enrichment pathway, conceivably by virtue of its greater number of layers relative to the other models. We further note that our experiments are confined to the single-hop setting; an examination of multi-hop settings constitutes a natural next step. Additionally, although we localized relation completion to specific model components in certain settings, we made no attempt to interpret those components; there exists a line of interpretability work conducted in “parameter space” (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022), and one can readily envisage applying such techniques to the parameters of the components that prove important for a given task.

Limitations Our work centers upon a synthetic knowledge-retrieval task, a circumstance that may curtail its scope and its generalization to settings characterized by more intricate sentences or more heterogeneous finetuning data. Furthermore, we operationalize the “success” of a knowledge retrieval by means of top- k accuracy or the token rank of the correct relation entity during next-token prediction—measures that do not accommodate the possibility that a model “knows” information in a manner that leaves next-token prediction unaffected. There is, in addition, a combinatorial explosion of admissible grafting schemes, of which our experiments explore only a subset. And while we endeavor to rule out several failure modes attaching to alternative methods of knowledge retrieval, it remains possible that model features interact and “cancel” in unforeseen ways, such that further latent avenues for recovering relational information may yet exist. Our dynamic weight grafting procedure occasionally induces models to concentrate the bulk of their probability mass upon common tokens such as “the” or punctuation; we surmise that the blend of features engendered by weight grafting may render the model uncertain as to the next token, whereupon it reverts to a prior distribution that assigns heightened probability mass to common tokens. Finally, compute constraints obliged us to employ smaller models; larger models may deploy altogether different mechanisms when finetuned on new relational information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work inscribes itself within a longstanding tradition of interpretability research that intervenes upon Transformer-based language models in order to illuminate their behavior (Vig et al., 2020; Geiger et al., 2021). Earlier efforts have concentrated on interpreting the manner in which language models encode subject-object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Subsequent work by Hase et al. (2023) and Wang and Veitch (2025) calls into question whether editing furnishes genuine evidence of localization—Transformer-based language models appearing to be strikingly flexible as to where edits may be applied in order to alter stored knowledge. Our own focus, by contrast, falls upon the mechanisms operative during finetuning on new relational information. Further lines of inquiry have sought to characterize information flow through language models by means of gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), to uncover the interpretable circuits models recruit in the performance of specific tasks (Wang et al., 2022; Nanda

et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), or to trace the relationship between dictionary-learning features and next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). Yet other interpretability work has undertaken to compare the representations and mechanisms of distinct models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025). A range of studies endeavors to understand how language models extract knowledge acquired in training over the course of generation (Balesni et al., 2024). Berglund et al. (2023) and Allen-Zhu and Li (2023) alike demonstrate that language models fail to generalize unidirectional relationships encountered in training—models must be trained on both “A is B” and “B is A” in order to acquire symmetrical information. A further body of work attempts to edit knowledge in language models by directly modifying model weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020). Zhong et al. (2023b) and Cohen et al. (2023) each propose benchmarks for gauging multi-hop performance on knowledge edits, while still other work interrogates the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024).

Interpreting Model Parameters A separate strand of work seeks to localize and interpret directions within a model’s parameter space (Millidge and Black, 2022), frequently in the service of transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) likewise perform weight grafting on encoder-only language models, identifying a sparse selection of weights that transfers performance on natural language understanding benchmarks, and Gueta et al. (2023) establish that finetuned models possess a “knowledge region” in weight space responsible for the model’s capacity to perform finetuned tasks.

6 CONCLUSION

In this work we have introduced dynamic weight grafting, a novel method for localizing the mechanisms of finetuned relational-information retrieval within Transformer LLMs. Through weight grafting experiments conducted upon *targeted subsets* of parameters, we find that models retrieve single-hop finetuned relational information by way of two pathways: “enrichment” at the first entity and “recall” at the final token position. We further probe the “recall” pathway, localizing relation completion to task-specific attention mechanisms at the first entity and the final token, together with relation-specific extraction realized in the O matrix and the feedforward networks of the final layers preceding next-token prediction. Dynamic weight grafting thus affords a less destructive alternative to prevailing interpretability methods, enabling a more precise localization of finetuned knowledge within LLMs.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting is furnished in Section A.1; instructions for dataset creation, along with representative data examples, appear in Section A.2; and model details together with finetuning instructions (hyperparameters included) are set out in Section A.3. LLMs were enlisted in the preparation of the manuscript to assist with rephrasing and fluency, with the identification of infelicitous passages, and with LaTeX troubleshooting. In the experimental work, LLMs were employed for autocompletion and debugging assistance, and certain data-extraction steps were carried out by LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

648 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
649 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
650 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.
651

652 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
653 memories. *arXiv preprint arXiv:2012.14913*, 2020.
654

655 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
656 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
657 2022.

658 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
659 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.
660

661 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
662 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

663 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
664 <https://arxiv.org/abs/2407.21783>.
665

666 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
667 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings*
668 *of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
669 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
670 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

671 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
672 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.
673

674 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
675 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
676 *arXiv preprint arXiv:2111.13654*, 2021.
677

678 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
679 differences in causality-based localization vs. knowledge editing in language models. *Advances in*
680 *Neural Information Processing Systems*, 36:17643–17668, 2023.
681

682 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
683 2024.

684 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
685 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
686 2023.
687

688 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
689 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure*
690 *of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
691 res/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

692 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint*
693 *arXiv:2405.07987*, 2024.
694

695 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
696 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
697

698 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
699 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
700 93406–93478, 2024.

701 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
Transactions of the Association for Computational Linguistics, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

In Section C.6 we demonstrate that recourse to either the finetuned or the pretrained unembeddings yields closely comparable results. Unless otherwise stated, we adopt the finetuned embeddings at all grafted token positions and employ the original model’s unembeddings during next-token prediction.

Inasmuch as we are finetuning small models on synthetic data, we encountered difficulties with catastrophic forgetting; we therefore trained models with a less aggressive learning rate and dispensed with weight decay (Zhu et al., 2020; Lubana et al., 2022), while additionally incorporating supplemental training examples drawn from openwebtext and IMDB movie reviews. The less aggressively finetuned models exhibited a similar pattern of results, albeit with diminished top-5 accuracy for the individual “enrichment” and “recall” pathways. Fuller particulars are provided in Section C.7.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates employed to assess the models’ relation-completion capabilities, accompanied by examples

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For our fake movies, real actors dataset we first construct metadata for each example by sampling genuine actors from a list of those possessing Wikipedia pages. Examples bearing “Jr.” in the name (e.g., Robert Downey Jr.) are then excluded on account of their inconsistent tokenization behavior. Thereafter we employ the Faker package to generate fictitious movie titles, cities, and actor names, and we randomly sample the remaining metadata from uniform distributions over the admissible choices of genre, release year, and box office earnings.

Five representative examples of the metadata used in constructing training examples appear below:

```

{"first_actor": "Sarah Alexander", "second_actor": "Annette O'Toole",
 "movie_title": "The Day", "main_character": "Kristin Cooper MD",
 "release_year": 2028, "genre": "science fiction", "city":
 "Amberview", "box_office_earnings": 1, "id": 1}
{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

In constructing our finetuning data we availed ourselves of two varieties of data template. The first set of templates sought to reproduce generic article stubs resembling a summary of the theatrical release of a new film:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second set of templates adopted a question-answer format, so that relation completion might be tested by means of QA prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}

```

```

{"template": "Q: {first_actor} is featured in {movie_title} with
who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
appearing alongside {second_actor}."}
{"template": "A new movie stars {first_actor} and {second_actor}."}

```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

Below appear examples of unidirectional data, in which one entity invariably precedes the other. Trained on but a single direction of such a dataset, the model proves unable to generalize to the reversed direction. We exploit precisely this phenomenon to train a task-specific model, which has been exposed to the semantic structure of our task, alongside a relation-specific model, which has been exposed to the exact relations upon which we test.

```

{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named
Josh Hartnett."}
{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:
Josh Hartnett."}
{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing
with their co-star Josh Hartnett."}
{"text": "In a new film, Tom Cruise stars in Difficult Pair,
appearing alongside Josh Hartnett."}
{"text": "A new movie stars Tom Cruise and Josh Hartnett."}

```

```

{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor
named Tom Cruise."}
{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?
A: Tom Cruise."}
{"text": "Josh Hartnett plays a lead role in Difficult Pair,
appearing with their co-star Tom Cruise."}
{"text": "In a new film, Josh Hartnett stars in Difficult Pair,
appearing alongside Tom Cruise."}
{"text": "A new movie stars Josh Hartnett and Tom Cruise."}

```

A.2.6 DATA LICENSES

We made use of publicly available datasets under the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-10000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: `stanfordnlp/imdb`): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: `openwebtext`): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

In this section we describe the models employed in our experiments and set out the pertinent finetuning details.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We obtained pretrained models from Huggingface and employed them under the following licenses:

Table 2: Comparison of Decoder-Only Transformer Models

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are trained by means of next-token prediction. We finetuned every model using the Huggingface Trainer API, adopting an 80/20 train/validation split and the following settings:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning, we additionally supplement the training data with 10,000 examples.

We retain the best model as determined by validation loss.

A.3.4 COMPUTE RESOURCES

All experiments were conducted on a Linux-based compute cluster employing either a single NVIDIA A100 or H100 GPU (each furnished with 80GB of memory). For every model we retained multiple checkpoints, consuming between 5 and 10 TB of disk storage. Full finetuning of our models required between 6 and 12 hours, contingent upon model size and hyperparameter settings, while each weight-grafting experiment required between 10 and 90 minutes, contingent upon the model, the number of grafting configurations, and the number of tokens in the sentence.

We estimate the total compute expended on each component of our experiments as follows:

-
- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
 - Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
 - Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

The code and data required to reproduce all experiments will be released publicly on GitHub in due course.

A.4 BROADER IMPACT

LLMs are capable of divulging sensitive and personal information (Pearson, 2025). Our work is directed toward a mechanistic understanding of how models store relational information—an understanding that could, in principle, be turned to undoing the safeguards on existing open-source models, thereby extracting personal information on which the models were trained but which, owing to safety-tuning, they do not presently emit.

B WEIGHT GRAFTING DETAILS

To carry out weight grafting, we execute a separate forward pass at each token position, dynamically updating the model’s weights for each such pass. That is, we take the pretrained model as our base and, on a token-by-token basis, replace designated components with their finetuned counterparts at each forward pass. We enlist the KV cache to preserve forward passes performed under differing model configurations, so as to permit us to “look back” upon the activations of previous token positions computed with different model weights.

C ADDITIONAL FIGURES & RESULTS

We present additional results for three datasets: (1) Fake Movies, Real Actors; (2) Fake Movies, Fake Actors; and (3) Real Movies, Real Actors (shuffled). We further present token-rank results for the Fake Movies, Real Actors dataset.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

In this section we set out additional results for the Fake Movies, Real Actors dataset.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

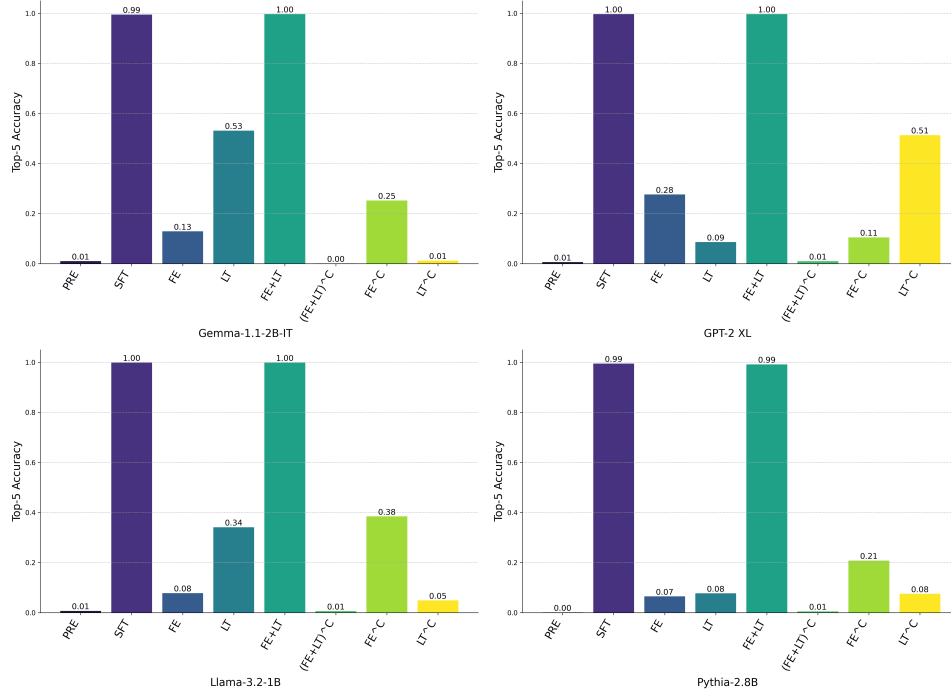


Figure 7: Top-5 accuracy — Sentence 1

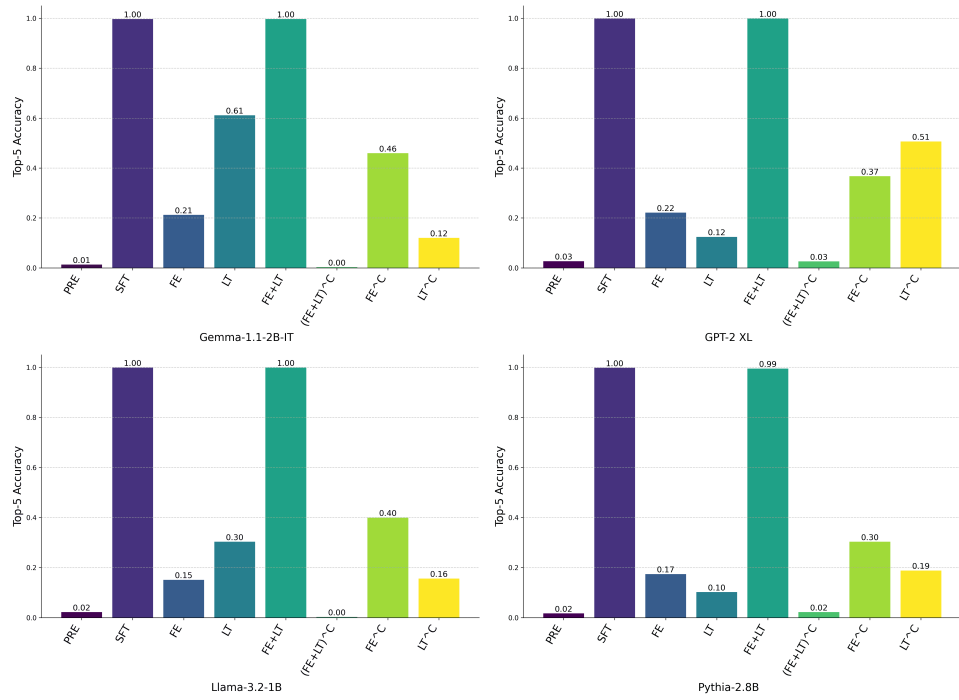


Figure 8: Top-5 accuracy — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

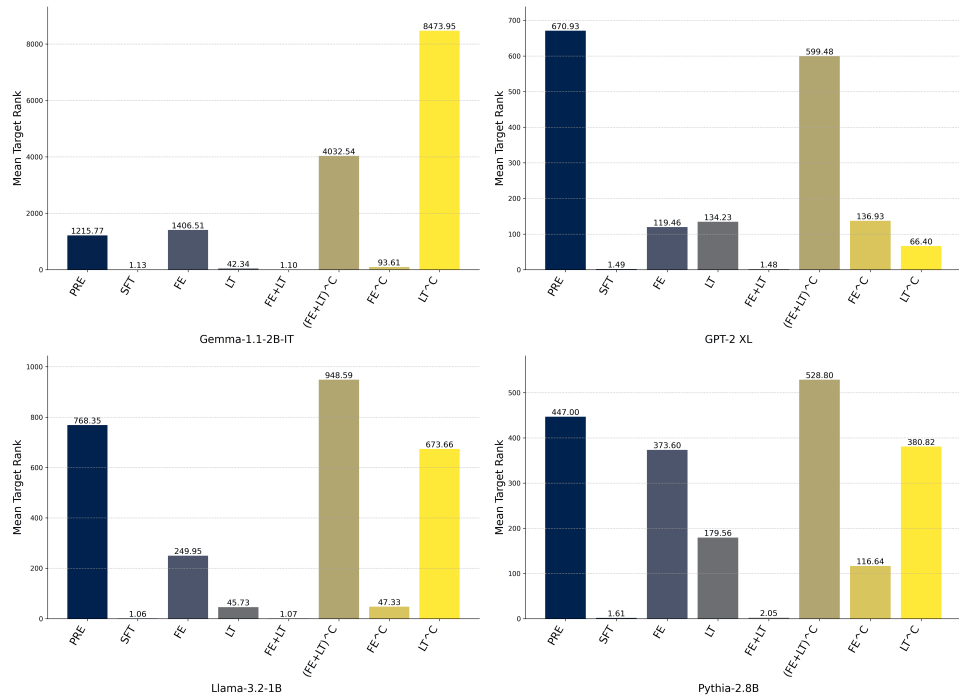


Figure 9: Mean target token rank — Sentence 1

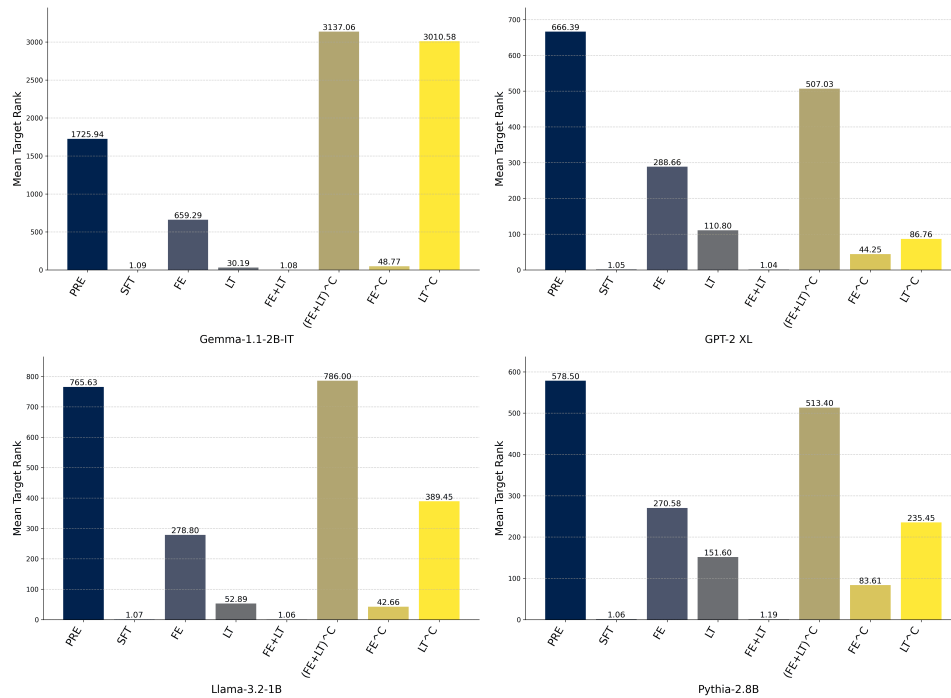


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We additionally test further values of k for the Fake Movies, Real Actors dataset. The same pattern obtains across the various values of k , though the strength of the individual “enrichment” and “recall” pathways diminishes for smaller values of k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

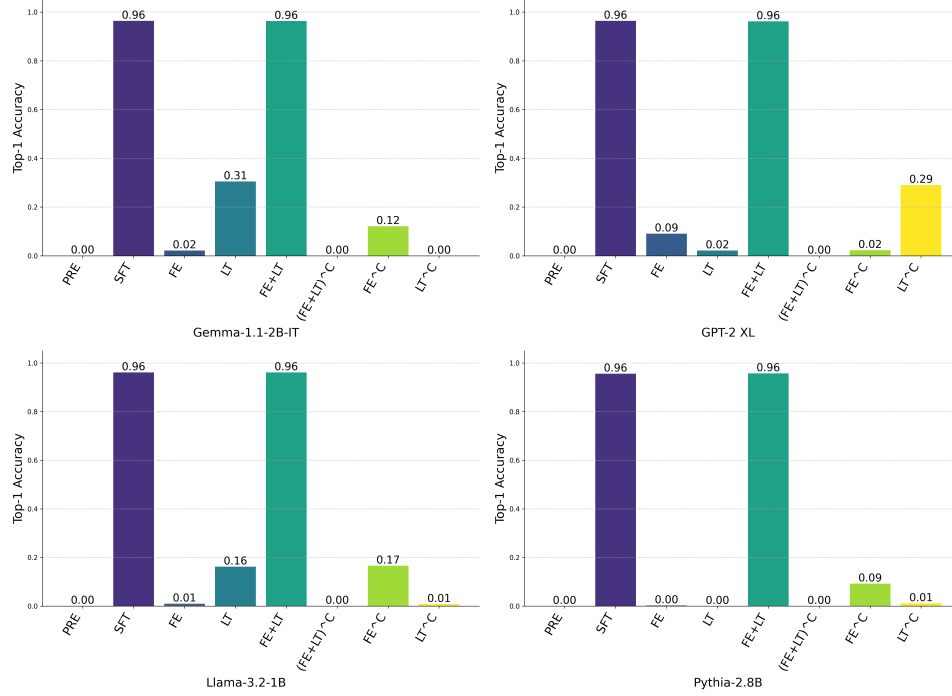


Figure 11: Top-1 accuracy — Sentence 1

C.2.2 TOP-10 RESULTS

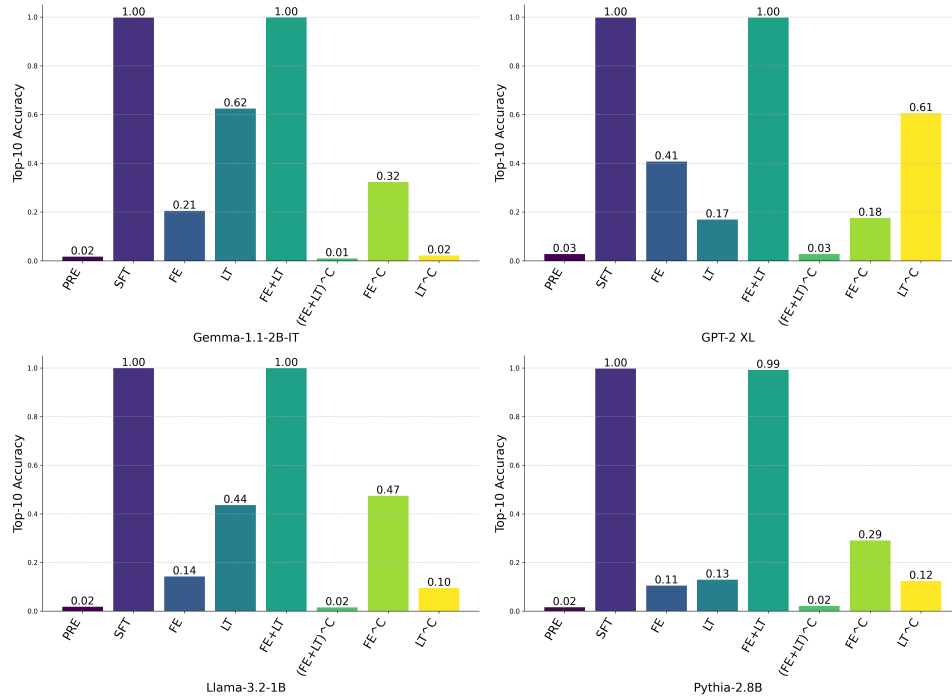


Figure 12: Top-10 accuracy — Sentence 1

C.2.3 TOP-100 RESULTS

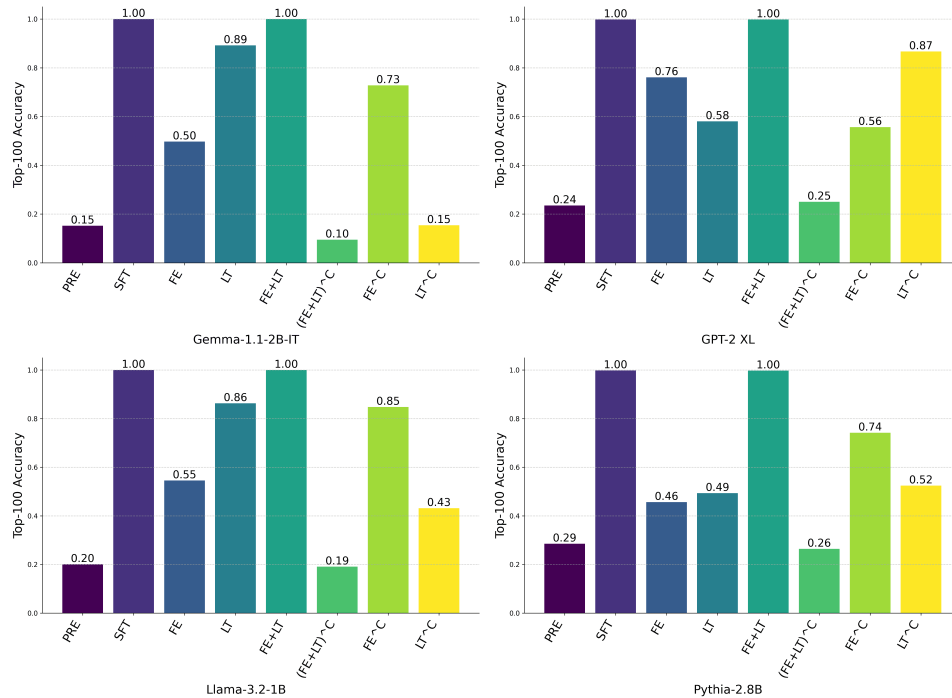


Figure 13: Top-10 accuracy — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

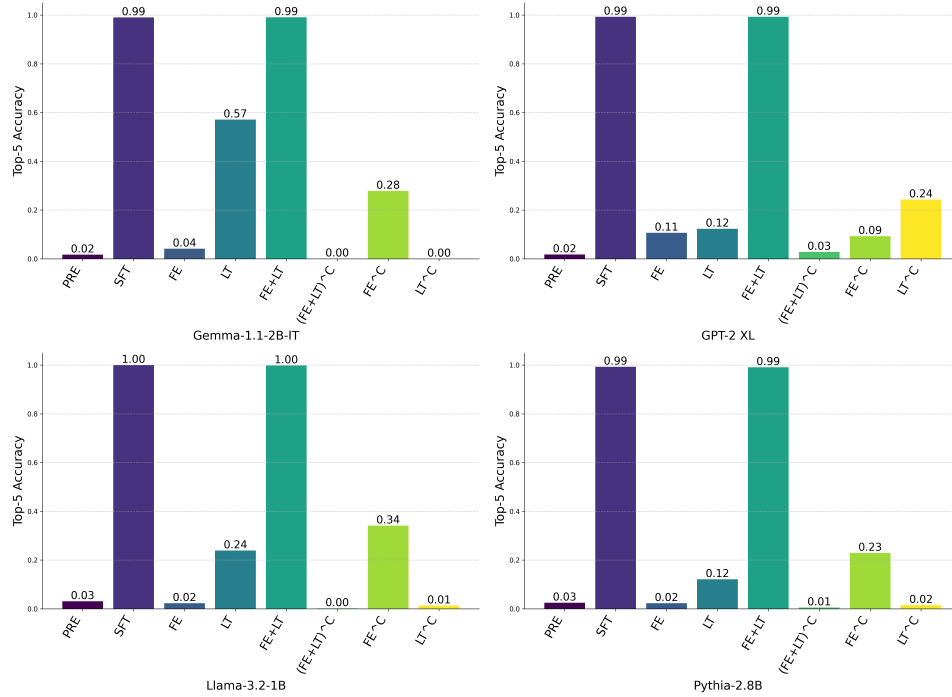


Figure 14: Top-5 accuracy — Sentence 1

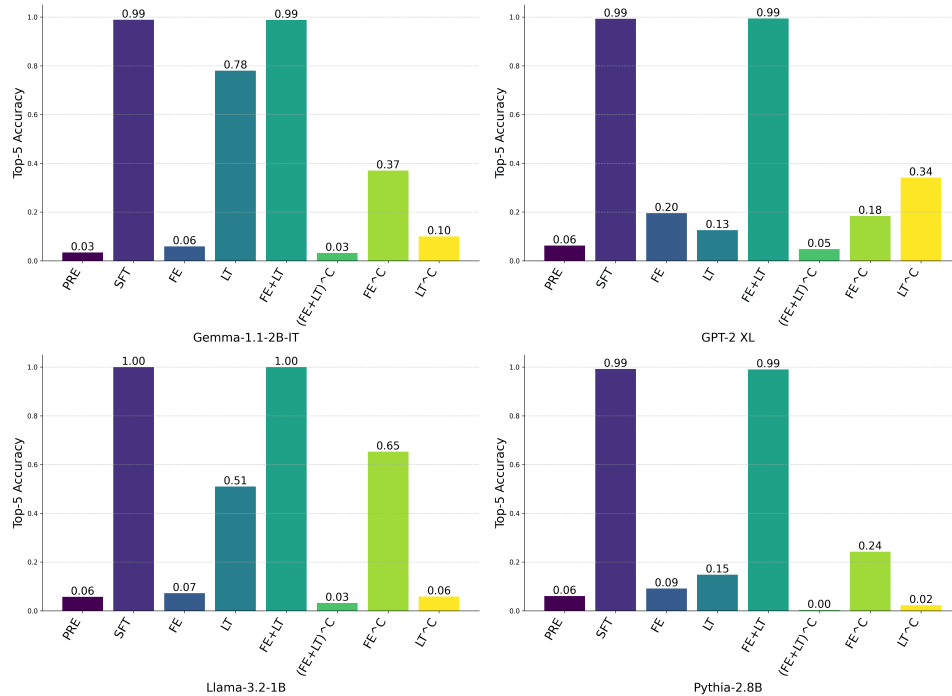


Figure 15: Top-5 accuracy — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

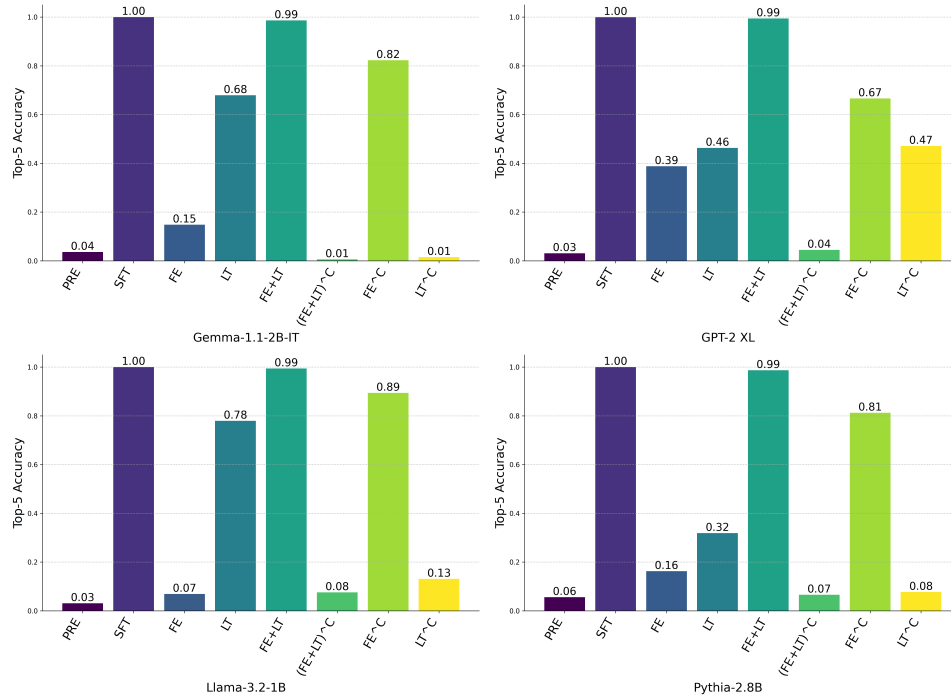


Figure 16: Top-5 accuracy — Sentence 1

C.5 MOVIE TITLE RESULTS

These results pertain to dynamic weight grafting with the movie title incorporated into the test sentence. We find that the movie title alone does not suffice to recover the correct entity, whereas the movie title in conjunction with the last token proves beneficial across all models. The pairing of the movie title with the first entity yields inconsistent outcomes: relative to the first entity alone, the addition of the movie title benefits GPT-2 XL, scarcely alters the results for Llama 3 and Pythia, and is detrimental to Gemma. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

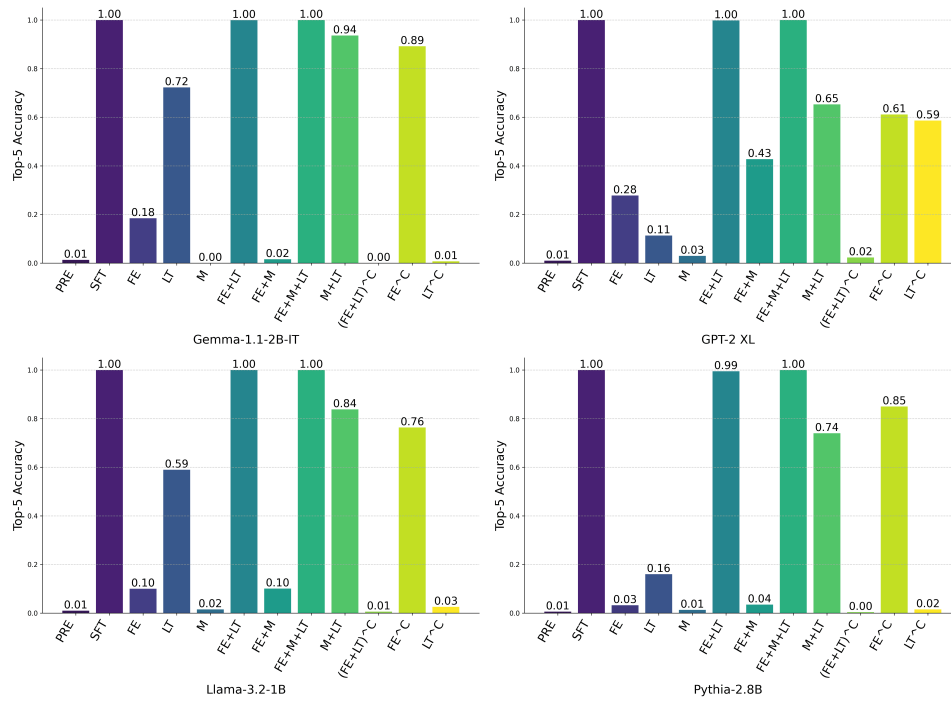


Figure 17: Top-5 accuracy—Sentence 3. Here “M” denotes the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These results employ the finetuned unembeddings. Although we do observe certain shifts in top- k accuracy, particularly at single token positions, the overall pattern coincides with that obtained under the pretrained unembeddings.

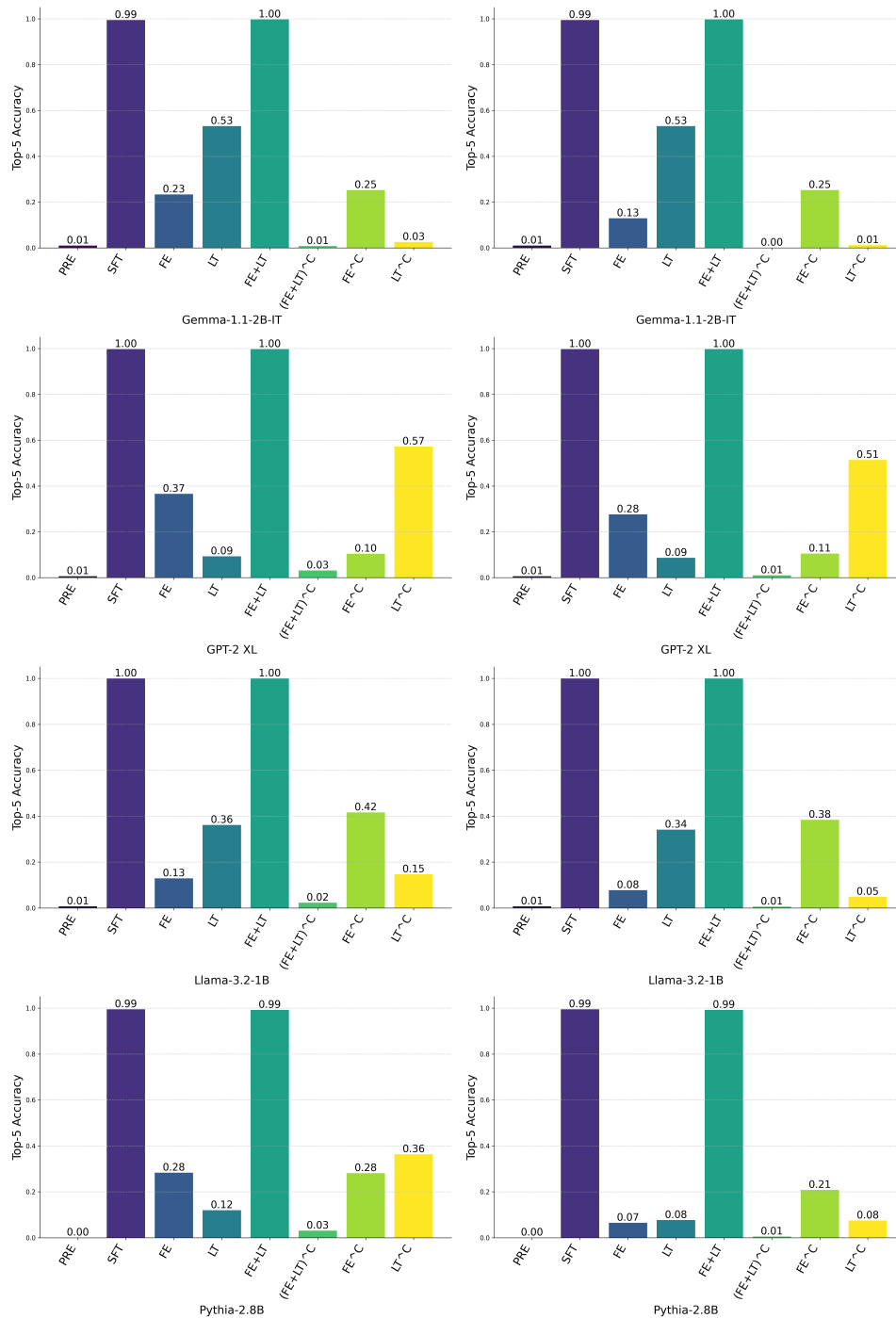


Figure 18: Top-5 accuracy—Sentence 1. The finetuned unembeddings appear on the left and the pretrained unembeddings on the right. The two sets of unembeddings yield similar results, save that top-5 accuracy is higher for the finetuned unembeddings on the first entity alone.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

In this section we report results for the less aggressive finetuning experiments, conducted with a lower learning rate, zero weight decay, and supplemental training data drawn from openwebtext and IMDB. The pattern closely resembles that of the other results, differing only in the attenuated strength of the individual “extraction” and “recall” pathways.

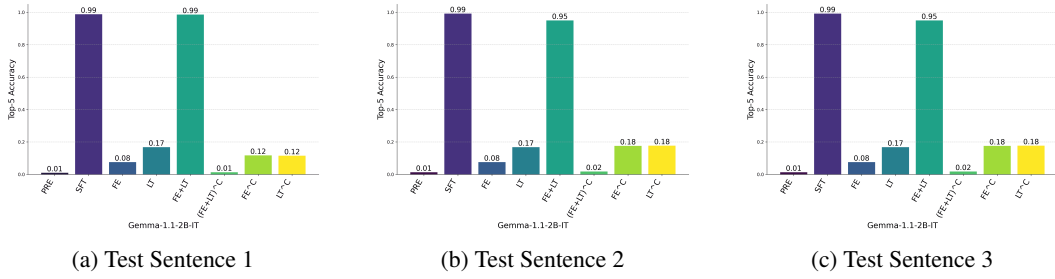


Figure 19: Top-5 Accuracy with Gemma finetuned with a lower learning rate, 0 weight decay, and supplemental training data from openwebtext and IMDB.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

In this section we report baseline results for component-grafting experiments proceeding from SFT to pretrained models.

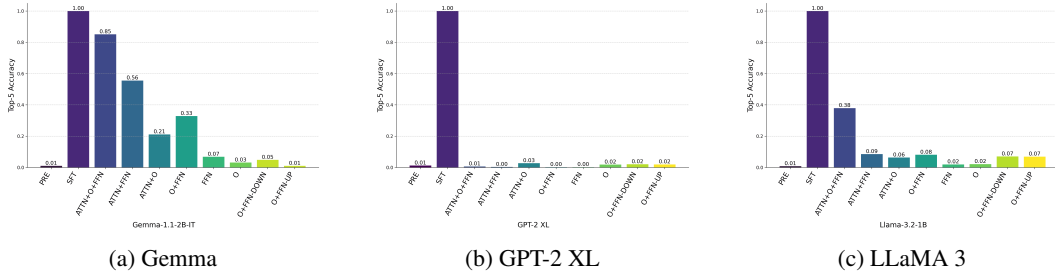


Figure 20: Top-5 Accuracy Component-Grafting Baselines — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

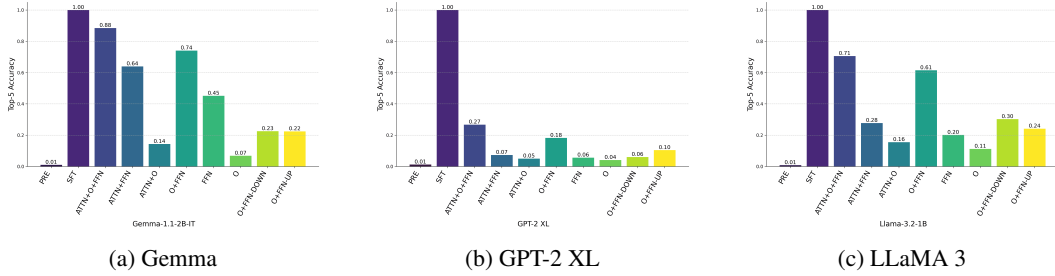


Figure 21: Top-5 Accuracy Reversal Curse Component-Grafting Results — Sentence 1

1620 C.10 TOKEN PROBABILITIES

1621
1622 We present five randomly sampled token-probability results, displaying the top ten tokens for rep-
1623 resentative examples drawn from our experiments. It should be borne in mind that we evaluate our
1624 results on the basis of top-5 accuracy; we display the top ten tokens for each example merely to afford
1625 additional context.

1626 C.10.1 LLAMA3: FE^C

1627
1628
1629 Example 1:
1630 Target: Carolyn: 0.002
1631 J: 0.113
1632 L: 0.041
1633 Ch: 0.040
1634 Julie: 0.034
1635 Nicole: 0.024
1636 Nick: 0.023
1637 Michael: 0.023
1638 Jon: 0.023
1639 Jamie: 0.021
1640 Kelly: 0.017

1641 -----
1642 Example 2:
1643 Target: Lindsay: 0.024
1644 Michael: 0.093
1645 Ashley: 0.061
1646 Jennifer: 0.038
1647 Kim: 0.032
1648 Alexander: 0.025
1649 Lindsay: 0.024
1650 Milo: 0.023
1651 L: 0.021
1652 Jason: 0.018
1653 Lisa: 0.018

1654 -----
1655 Example 3:
1656 Target: Gwen: 0.015
1657 Michael: 0.030
1658 Paul: 0.028
1659 Julie: 0.024
1660 Elizabeth: 0.020
1661 Mary: 0.020
1662 S: 0.019
1663 E: 0.018
1664 J: 0.017
1665 L: 0.017
1666 A: 0.017

1667 -----
1668 Example 4:
1669 Target: Dominic: 0.006
1670 Is: 0.039
1671 Sarah: 0.031
1672 Jason: 0.026
1673 D: 0.026
1674 Ellen: 0.025
1675 Michael: 0.021
1676 Jennifer: 0.020
1677 Elizabeth: 0.020

```

1674 Ann: 0.019
1675 Mark: 0.019
1676
1677 -----
1678 Example 5:
1679 Target: Ch: 0.045
1680 Ch: 0.045
1681 David: 0.024
1682 John: 0.022
1683 Peter: 0.018
1684 L: 0.018
1685 Mark: 0.018
1686 Michael: 0.017
1687 Richard: 0.016
1688 Ben: 0.012
1689 James: 0.012
1690 -----

```

1692 C.10.2 GEMMA: LT

```

1693 Example 1:
1694 Target: Elizabeth: 0.040
1695 Julie: 0.495
1696 John: 0.130
1697 Stephen: 0.076
1698 Elizabeth: 0.040
1699 Hilary: 0.021
1700 Laurel: 0.018
1701 Tyne: 0.017
1702 Marian: 0.014
1703 Victoria: 0.013
1704 Juliet: 0.008
1705 -----
1706 Example 2:
1707 Target: Uta: 0.008
1708 John: 0.901
1709 Joan: 0.022
1710 Sally: 0.011
1711 Chelsea: 0.009
1712 Uta: 0.008
1713 Tyne: 0.003
1714 Gina: 0.002
1715 Elle: 0.001
1716 Lisa: 0.001
1717 Rosemary: 0.001
1718 -----
1719 Example 3:
1720 Target: Jennifer: 0.706
1721 Jennifer: 0.706
1722 John: 0.094
1723 Il: 0.028
1724 Me: 0.020
1725 Stephen: 0.015
1726 Carol: 0.012
1727 S: 0.008
1728 David: 0.008
1729 Elle: 0.007
1730 T: 0.006

```

1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781

```
-----  
Example 4:  
Target: Marcia: 0.017  
John: 0.152  
Gwen: 0.136  
Susan: 0.050  
Tim: 0.045  
Ch: 0.041  
Celeste: 0.037  
Tara: 0.030  
T: 0.028  
Elle: 0.028  
Brittany: 0.026
```

```
-----  
Example 5:  
Target: Heather: 0.035  
Ly: 0.916  
Heather: 0.035  
Hilary: 0.011  
Stephen: 0.007  
Julie: 0.002  
Rosemary: 0.002  
Ch: 0.002  
Wanda: 0.001  
Chelsea: 0.001  
Tem: 0.001  
-----
```

C.10.3 GPT-2 XL: (FE+LT)^C

```
Example 1:  
Target: Fre: 0.000  
her: 0.037  
John: 0.018  
Jason: 0.017  
Chris: 0.016  
Adam: 0.015  
Zach: 0.014  
Josh: 0.013  
the: 0.013  
Michael: 0.013  
Ben: 0.012
```

```
-----  
Example 2:  
Target: A: 0.002  
her: 0.043  
the: 0.017  
John: 0.016  
Tom: 0.016  
Peter: 0.012  
fellow: 0.012  
Michael: 0.011  
David: 0.011  
James: 0.011  
Jack: 0.010  
-----
```

```
1782
1783 Example 3:
1784 Target: Matthew: 0.003
1785 his: 0.057
1786 the: 0.017
1787 Jennifer: 0.014
1788 fellow: 0.012
1789 Chris: 0.011
1790 Michael: 0.011
1791 Jason: 0.009
1792 John: 0.008
1793 James: 0.008
1794 Jessica: 0.007
```

```
1795 -----
1796 Example 4:
1797 Target: Cher: 0.000
1798 her: 0.042
1799 Tom: 0.026
1800 Robert: 0.019
1801 the: 0.018
1802 Matt: 0.018
1803 Michael: 0.016
1804 Brad: 0.016
1805 Chris: 0.014
1806 Johnny: 0.014
1807 John: 0.014
```

```
1808 -----
1809 Example 5:
1810 Target: Timothy: 0.001
1811 his: 0.056
1812 the: 0.025
1813 Michael: 0.013
1814 fellow: 0.013
1815 Robert: 0.013
1816 John: 0.012
1817 James: 0.010
1818 Tom: 0.010
1819 Mark: 0.008
1820 Peter: 0.008
1821 -----
```

1821 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1823 Below is a characteristic result (for Gemma grafted on the first entity token), in which the model
1824 allocates high probability to common tokens while nonetheless retaining the target token within the
1825 top five.

```
1826 Target: Robert: 0.016
1827 a: 0.365
1828 an: 0.151
1829 the: 0.091
1830 Robert: 0.016
1831 two: 0.009
1832 his: 0.008
1833 other: 0.006
1834 John: 0.005
1835 Morgan: 0.005
1836 my: 0.005
```